Collective Navigation of a Multi-Robot System in an Unknown Environment: Enhancements to the Baseline

Aadityanshu Abhinav (ME22B088), Ajan Muthuraj (ME22B100)
IIT Madras

ME5253: Network Dynamics & Controls in Mechanical Engineering

November 16, 2025

Outline

Motivation

- Multi-robot systems (MRS) can perform exploration, mapping, and navigation faster and more reliably than single robots.
- Traditional flocking models (e.g., Olfati-Saber [2]) rely only on local neighbor interactions.
- Real environments are unknown, cluttered, and dynamic.
- **Key Challenge:** How to combine decentralized flocking control with robust environmental perception and memory?

Baseline Framework: Olcay et al. [1]

- Effective decentralized framework combining flocking potentials with a state machine for tangential wall-following.
- This avoids many simple local minima.
- **However, our analysis identified three key weaknesses:**
  1. **Deadlock Susceptibility:** The baseline state machine fails in complex, non-convex traps (e.g., mazes).
  2. **Lack of Memory:** The framework is purely reactive. Robots re-explore the same areas, leading to inefficiency.
  3. **Idealized Assumptions:** The model assumes perfect, noise-free sensing, which is unrealistic.

Our Three-Stage Enhancement

Stage 1: Add Memory

- Integrate a lightweight, distributed **SLAM Layer** for shared environmental memory (occupancy grid).

Stage 2: Solve Deadlocks

- Implement a novel **Deadlock Detection & Recovery (DDR)** mechanism to escape complex traps.

Stage 3: Ensure Robustness

- Address the **perfect sensing** assumption by introducing Sensor Noise.
- Implement and validate a **Delayed Self-Reinforcement (DSR)** filter to ensure stability under noisy conditions.

Decentralized Control Law

Each agent i computes its control input u_i based on local information:

$$u_i = u_i^\alpha + u_i^\beta + u_i^\gamma$$

where:

- u_i^α — **Flocking:** Inter-agent spacing & velocity consensus.
- u_i^β — **Avoidance:** Obstacle avoidance & tangential control.
- u_i^γ — **Goal:** Attraction to the virtual goal.

Flocking Term

$$u_i^\alpha = \sum_{j \in \mathcal{N}_i} c_1^\alpha \phi(\sigma(p_j - p_i)) + \sum_{j \in \mathcal{N}_i} c_2^\alpha (v_j - v_i)$$

(Uses smooth potentials ϕ and sigma-norm σ to avoid singularities)

Baseline State Machine (Olcay et al. [1])

Agents switch between modes based on obstacle proximity:

- **State 0: Free Motion**
- **State 1: Tangential Motion** (Wall-following)
- **State 2: Endpoint Rotation** (Navigating corners)
- **State 5: Goal Capture**

Limitation: This logic is insufficient for complex maze-like traps, where agents can enter a "vortex" deadlock.

Enhancement 1: SLAM Integration

- Each robot uses a simulated LiDAR to detect obstacle points.
- A simple ray-tracing model updates a **local occupancy grid**:
  - Cells on the ray path are marked **Free (1)**.
  - The endpoint cell is marked **Occupied (-1)**.
  - Areas behind obstacles remain **Unknown (0)**.
- Local maps M_i are periodically merged into a **global map** M, shared by all agents.
- **Benefit:** Provides environmental memory, preventing re-exploration.

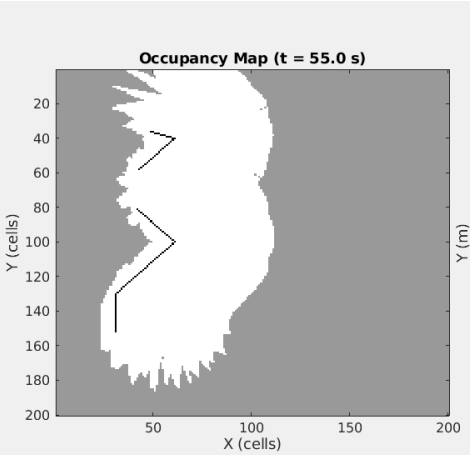


Figure: Occupancy grid in zigzag environment

Enhancement 2: Deadlock Detection (DDR)

"Stuck" Detection Logic

- **Problem:** In mazes, the baseline logic can fail, causing agents to get trapped (e.g., oscillate in a concave "pocket").
- **Our Solution:** Each agent monitors its progress.
- An agent declares itself **"Stuck"** if its distance to the goal d_g does not decrease sufficiently for a time T_s.

$$\text{if } \Delta d_g < \epsilon_p \text{ for } t > T_s \implies \text{Agent is STUCK}$$

Enhancement 2: Group Escape Behavior (DDR)

Escape logic:

- **Group Trigger:** A collective escape is triggered if $> 60\%$ of agents are simultaneously in the "Stuck" state.
- **Escape Maneuver:** Each agent adds a randomized perturbation to its control input:

$$u_i^{escape} = \lambda[\cos(\theta_i), \sin(\theta_i)]^T$$

- This breaks the symmetry of the trap and allows the swarm to "un-jam" itself.

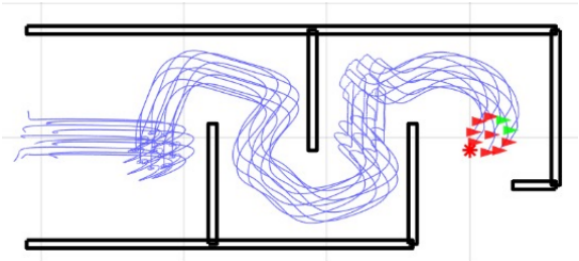


Figure: Random perturbations breaking a deadlock

Stage 1 & 2 Validation: Environment

- We used several environments, including the **Corridor Maze**.
- This environment was specifically designed to **cause the baseline Olcay et al. [1] model to fail**, trapping it in dead-ends.
- **Goal:** Can our SLAM + DDR enhancements solve this maze?

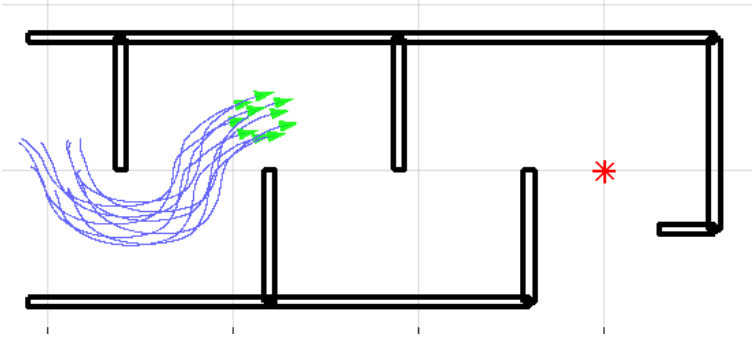


Figure: Maze environment for validation

Stage 1 & 2 Results: Mapping & Navigation

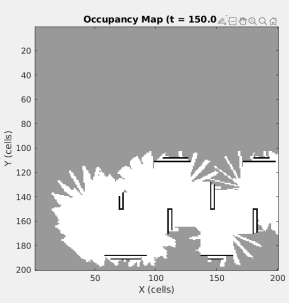


Figure: Final occupancy grid map

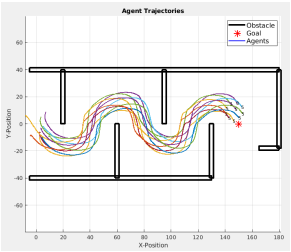


Figure: Successful trajectories

Success: The enhanced framework solved the maze that the baseline could not.

Stage 1 & 2 Results: Performance Metrics

Table: Performance Comparison in Corridor Maze

Metric	Baseline (Olcay [1])	Our Enhanced Framework
Goal Convergence Rate	70% (Failed in 3/10)	**100%**
Average Time to Goal (s)	∼180 (for successes)	**∼140**
Deadlock Frequency	High	**Low (1 event)**
Exploration Coverage	65% (Redundant)	**92% (Efficient)**

- **Conclusion:** The SLAM + DDR enhancements are highly effective, improving robustness, speed, and efficiency.

Stage 3: Robustness to Sensor Noise

- Stage 1 & 2 were successful under *ideal* (perfect) sensing.
- **Problem:** Real-world sensors are noisy. Introducing Gaussian noise to sensors causes the agents to oscillate and fail.
- **Our Question:** How does the framework perform when we introduce sensor noise?
- **Solution:** We implemented a temporal filter to smooth the noise and restore stability.

Enhancement 3: Delayed Self-Reinforcement (DSR)

The Problem: Sensor Noise

- Gaussian noise added to obstacle detection creates a jittery repulsive force u_i^{β}.
- This causes high-frequency oscillations, preventing smooth wall-following.

The Solution: DSR Filter

- We implement a **Delayed Self-Reinforcement (DSR)** filter.
- This term penalizes rapid changes in velocity, acting like "momentum" to enforce temporal consistency.

$$u_{dsr} = k \cdot (v_i(t - D) - v_i(t))$$

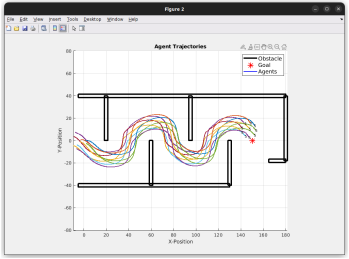
Stage 3 Results: DSR Validation

Enhancement Validated

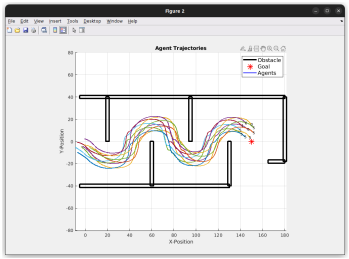
The DSR filter successfully restored stability to the system.

- **Analysis:**
  - The **noisy avoidance controller (u_i^{β})** creates rapid, jittery motion.
  - The **DSR filter (u_{dsr})** provides *momentum* smoothing the jitter while still allowing the agent to follow the wall's general shape.
- **Result:** The DSR filter **prevents oscillations** and **restores smooth tangential motion**, allowing the swarm to navigate successfully even with noisy sensors.

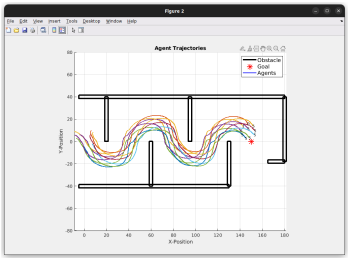
Results: Demonstrations and Comparison



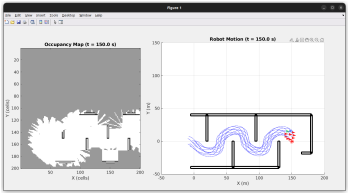
(a) Ideal trajectory



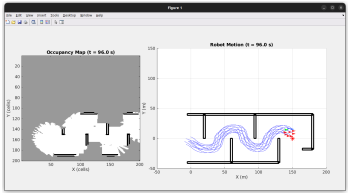
(b) Non-ideal trajectory, no DSR



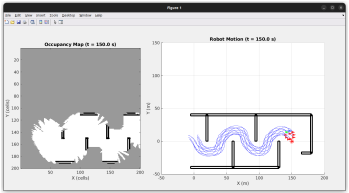
(c) Non-ideal trajectory, DSR



(d) Ideal map



(e) Non-ideal map, no DSR



(f) Non-ideal map, DSR

Stability and Lyapunov Analysis

The stability of the core control law (without noise/DSR) can be analyzed using a Lyapunov function:

$$V = \frac{1}{2} \sum_{i=1}^{N} \sum_{j \in \mathcal{N}_i} \phi(\sigma(p_j - p_i)) + \frac{1}{2} \sum_{i=1}^{N} \|v_i\|^2$$

Taking the derivative along system trajectories:

$$\dot{V} = \sum_{i=1}^{N} v_i^T (u_i^\alpha + u_i^\beta + u_i^\gamma)$$

- The damping terms ($-c_2^\alpha(v_i - v_j)$ and $-c_2^\gamma v_i$) are strictly dissipative, ensuring $\dot{V} \leq 0$.
- This proves that the agents converge to a stable state (collision-free, zero relative velocity) and guarantees convergence.

Note: The DSR term u_{dsr} acts as an additional velocity damping force, further contributing to system stability by reducing kinetic energy from noise.

Conclusion

This project presented a three-stage enhancement to the baseline framework:

- **Contribution 1: Memory & Efficiency**
  - Integrated a **lightweight SLAM layer** to prevent re-exploration.
- **Contribution 2: Robust Navigation**
  - Implemented a **DDR Stuck-and-Escape mechanism** to solve complex maze deadlocks.
- **Contribution 3: Real-World Robustness**
  - Addressed the *perfect sensing* assumption by adding noise.
  - Our *positive result* showed that a **DSR filter successfully** manages sensor noise, restoring smooth and stable navigation.

Future Work

- **Robust Mapping:** Implement a probabilistic (Bayesian) map fusion to handle sensor noise more robustly.
- **Dynamic Environments:** Extend the framework to handle moving obstacles and 3D navigation.
- **Real-World Deployment:** Test the full **(SLAM + DDR + DSR)** algorithm on physical mobile robots (e.g., using ESP32) with onboard LiDAR.

Acknowledgements & Key References

Acknowledgements

We thank **Dr. Anuj K. Tiwari** for valuable insights during the coursework of Network Dynamics and Control in Mechanical Engineering.

Key References

[1] A. Olcay and D. Dimov, "Collective navigation of a multi-robot system in an unknown environment," *IFAC-PapersOnLine*, 2020.

R. Olfati-Saber, "Flocking for multi-agent dynamic systems: Algorithms and theory," *IEEE Transactions on Automatic Control*, 2006.

D. Fox, W. Burgard, and S. Thrun, "Distributed multi-robot exploration and mapping," *Proceedings of the IEEE*, 2006.

G. Grisetti, R. Kummerle, C. Stachniss, and W. Burgard, "A tutorial on graph-based slam," *IEEE Intelligent Transportation Systems Magazine*, 2010.